ECx00G&EC600U&EG800G Series

QuecOpen(SDK) Quick Start Guide

LTE Standard Module Series

Version: 1.0

Date: 2023-09-12

Status: Released

At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.
Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China
Tel: +86 21 5108 6236
Email: info@quectel.com

Or our local offices. For more information, please visit:
http://www.quectel.com/support/sales.htm.

For technical support, or to report documentation errors, please visit:
http://www.quectel.com/support/technical.htm.
Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an "as available" basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

About the Document

Revision History

Version	Date	Author	Description
-	2023-07-30	Neo KONG	Creation of the document
1.0	2023-09-12	Kevin WANG	First official release

Contents

Table Index

Figure Index

1 Introduction

Quectel EC600U series, ECx00G family and EG800G series modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS, which is intended to simplify the design and development of IoT applications.

This document is applicable to QuecOpen® solution based on CSDK build environment. It introduces CSDK directory structure, compilation environment setup, application development and compilation process, feature removal, partition adjustment, firmware download and addressing FAQ for EC600U series, ECx00G family and EG800G series modules.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC600U Series
ECx00G	EC200G-CN
	EC600G Series
	EC800G-CN
-	EG800G Series

2 CSDK Directory Structure

The directory may differ depending on the actual released versions of the QuecOpen CSDK. The QuecOpen CSDK directory is shown in ***Figure 1***, with details in the following table.

Table 2: CSDK Directory Structure

Directory Name	Description
cmake, *prebuilts*, *tools*	Includes compilation tools, configuration files, and compilation scripts.
components/appstart	Includes source files of Kernel start program.
components/bootloader	Includes source files of boot loader.
components/hal	Includes partition and Flash configuration files.
components/ql-config	Includes the default Kernel firmware, original application firmware, and preset files of the module.
components/ql-kernel	Includes all open API header files and library files.
components/ql-application	Includes application reference routines and demos for various features.
components/libs, *components/net*, *components/newlib*	Includes header files and library files required by the application.
build_all.bat, *build_all.sh*	● Windows: Compiles applications in Windows CLI using the configured parameters. See ***Chapter 5*** for details. ● Linux: Compiles applications in Linux CLI using the configured parameters. Administrator mode is required during compilation in Linux. And the script should be executed using **bash**. See ***Chapter 5*** for details.
CmakeLists.txt	Top compilation configuration file.

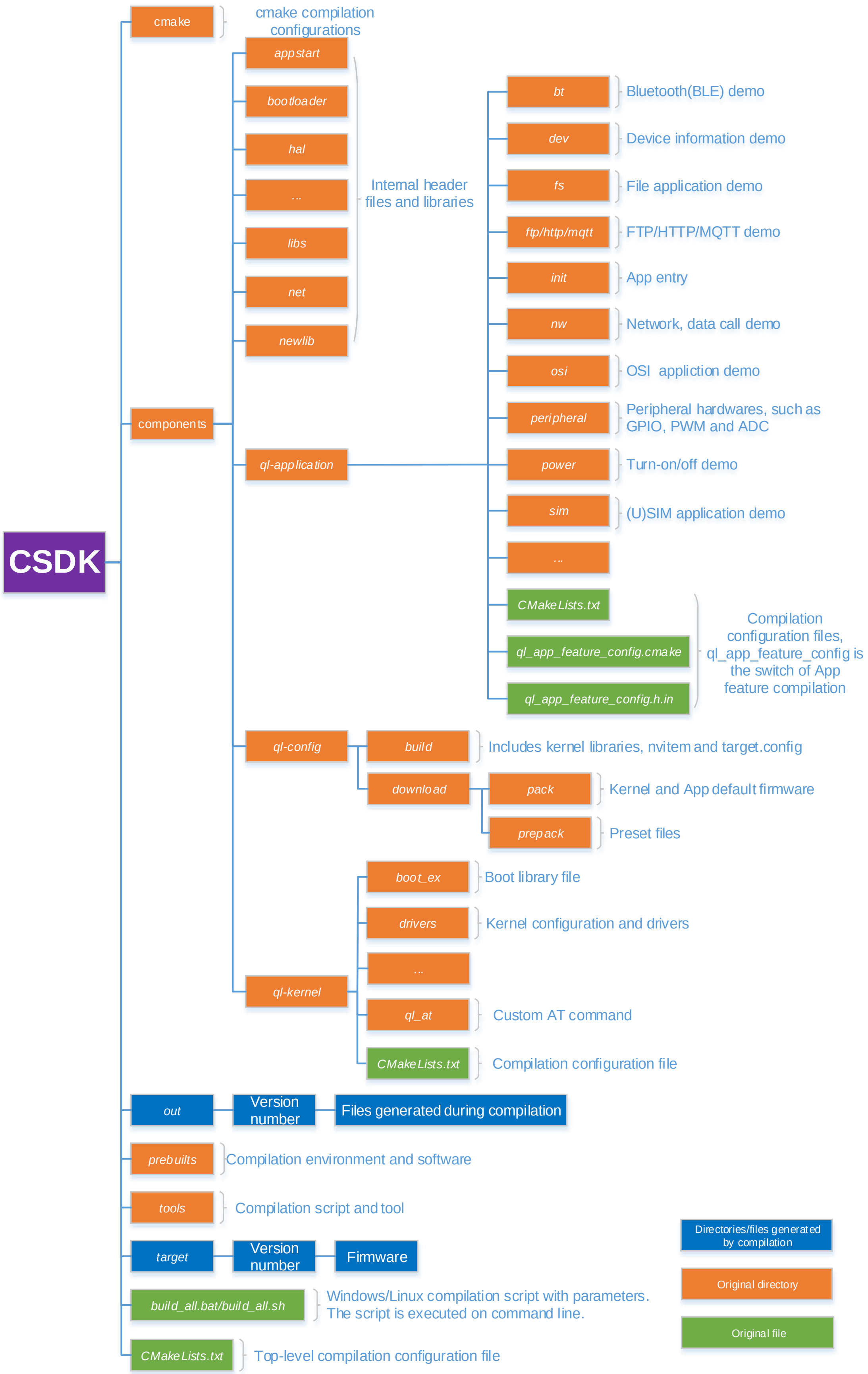


Figure 1: CSDK Directory Structure

3 Compilation Environment Setup

3.1. Host System

If the compilation environment operates on a Windows host system, ensure that the host system is either a 32-bit or 64-bit Windows 7/10 and install the following software:

- For 32-bit Windows: Visual C++ Redistributable for Visual Studio 2015 x86 or later versions
- For 64-bit Windows: Visual C++ Redistributable for Visual Studio 2015 x64 or later versions

If the compilation environment operates on a Linux host system, ensure that the operating system is either Ubuntu 16.04 or Ubuntu 20.04 and the language is Python 3.5 or later versions, and execute the following commands to install the required components:

```
sudo apt install build-essential python3 python3-tk qtbase5-dev
sudo apt install libc6:i386 libstdc++6:i386 zlib1g:i386
sudo apt install protobuf-compiler
```

3.2. Application Compilation Environment

The application compilation toolchain for EC600U series, ECx00G family and EG800G series modules utilize gcc-arm-none-eabi, which is the GCC toolchain of ARM bare-metal system. *LTE01R02XXXU* and *LTE01R03XXXU* modules utilize version 7.2.1, and *LTE01R05XXXU* and *LTE01XXXG* modules utilize version 9.2.1. The gcc-arm-none-eabi is integrated in the QuecOpen CSDK in the *prebuilts* directory.

When compiling an application, only the gcc-arm-none-eabi toolchain integrated in the QuecOpen CSDK is supported. The gcc-arm-none-eabi toolchain installed on the host system is not supported for this purpose.

4 Application Development

4.1. Application Demo Description

QuecOpen CSDK provides application demos in the *components\ql-application* directory in CSDK as a reference for application development.

The application entry point, *appimg_enter()*, is in the *ql_init.c* file in the *components\ql-application\init* directory of the QuecOpen CSDK. To begin, initiate an initialization thread within this function, and then call the initialization interface of each feature component in this initialization thread. Feature components are organized into folders. To incorporate a feature component, refer to the code snippet below.

```
static void ql_init_demo_thread(void *param)
{
    QL_INIT_LOG("init demo thread enter, param 0x%x", param);
    ql_osi_demo_init();

#ifdef QL_APP_FEATURE_FILE
    ql_fs_demo_init();
#endif

#ifdef QL_APP_FEATURE_MQTT
    ql_mqtt_app_init();
#endif
    ql_rtos_task_sleep_ms(10);
    ql_rtos_task_delete(NULL);
}

int appimg_enter(void *param)
{
    QlOSStatus err = QL_OSI_SUCCESS;
    ql_task_t ql_init_task = NULL;

    QL_INIT_LOG("init demo enter");
    prvInvokeGlobalCtors();

    err = ql_rtos_task_create(&ql_init_task, 1024, APP_PRIORITY_NORMAL, "ql_init", ql_init_demo_thread, NULL, 1);
    if(err != QL_OSI_SUCCESS)
    {
        QL_INIT_LOG("init failed");
    }
    return err;
}
```

Figure 2: Application Entry

4.2. Add Application Feature Component

This chapter explains how to add an application feature component using the audio Application example provided in the QuecOpen CSDK.

4.2.1. Create Directory for Application Feature Component

First, create a feature directory under the *components/ql-application* directory in the QuecOpen CSDK. Then place the source files, header files, and *CMakeLists.txt* compilation configuration file in the feature directory. Take an audio application for example, the directory structure is as follows and the *inc* folder is used to store header files:

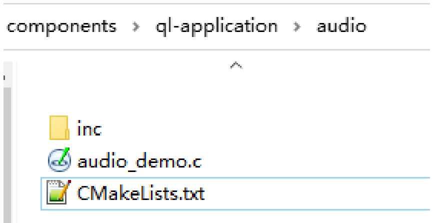


Figure 3: Example of Directory for Application Feature Component

4.2.2. Create an Application Thread

Use *ql_rtos_task_create()* to create the application thread of the feature component in *ql_audio_app_init()*, the initialization function of the feature component source code file, as shown in the following example:

```
static void ql_audio_demo_thread(void *param)
{
    QlOSStatus err = QL_OSI_SUCCESS;
    QL_AUDIO_LOG("audio demo thread enter, param 0x%x", param);

    for (int n = 0; n < 10; n++)
    {
        QL_AUDIO_LOG("hello audio demo %d", n);
        ql_rtos_task_sleep_ms(500);
    }

    err = ql_rtos_task_delete(NULL);
    if(err != QL_OSI_SUCCESS)
    {
        QL_AUDIO_LOG("task deleted failed");
    }
}

void ql_audio_app_init(void)
{
    QlOSStatus err = QL_OSI_SUCCESS;
    ql_task_t ql_audio_task = NULL;

    QL_AUDIO_LOG("audio demo enter");

    err = ql_rtos_task_create(&ql_audio_task, 1024, APP_PRIORITY_NORMAL, "ql_audio", ql_audio_demo_thread, NULL, 5);
    if(err != QL_OSI_SUCCESS)
    {
        QL_AUDIO_LOG("audio task create failed");
    }
}
```

Figure 4: Create Feature Component Application Thread

4.2.3. Modify Configuration File

Modify the *components\ql-application\audio\CMakeLists.txt* application configuration file, if needed, as follows:

```
set(target ql_app_audio)   Library name

add_library(${target} STATIC)
set_target_properties(${target} PROPERTIES ARCHIVE_OUTPUT_DIRECTORY ${out_app_lib_dir})
target_compile_definitions(${target} PRIVATE OSI_LOG_TAG=LOG_TAG_QUEC)
target_include_directories(${target} PUBLIC inc)   Header file path
#target_link_libraries(${target} PRIVATE ql_api_common)

target_sources(${target} PRIVATE
    audio_demo.c                Source file list, supporting relative path
)

relative_glob(srcs include/*.h src/*.c inc/*.h)
beautify_c_code(${target} ${srcs})
```

Figure 5: Modify Configuration File

4.2.4. Add Compilation Task

Modify the *components\ql-application\CMakeLists.txt* configuration file to add the application feature component directory to the file to compile the directory when compiling the firmware package:

```
# Copyright (C) 2020 QUECTEL Technologies Limited and/or its affiliates("QUECTEL").
# All rights reserved.
#

add_subdirectory_if_exist(init)

add_subdirectory_if_exist(nw)

add_subdirectory_if_exist(peripheral)

add_subdirectory_if_exist(osi)

add_subdirectory_if_exist(dev)

add_subdirectory_if_exist(sim)

add_subdirectory_if_exist(power)

if(QL_APP_FEATURE_FILE)
add_subdirectory_if_exist(fs)
endif()

if(QL_APP_FEATURE_AUDIO)
add_subdirectory_if_exist(audio)
if(QL_APP_FEATURE_AUDIO_TTS)
add_subdirectory_if_exist(tts)
endif()
endif()
```

Figure 6: Add Compilation Tasks

4.2.5. Link Application Feature Component

Modify the *CMakeLists.txt* configuration file in the *components/ql-application/init* directory to add the name of the library that links the new application feature components. If a feature component compilation switch is added for the application, the option switch control needs to be added to the *CMakeLists.txt* file as well, as shown in the following example:

```
if(CONFIG_APPIMG_LOAD_FLASH)
    set(target ${QL_APP_BUILD_VER})
    add_appimg_flash_ql_example(${target} ql_init.c)

    target_link_libraries(${target} PRIVATE ql_app_nw ql_app_peripheral ql_app_osi ql_app_dev ql_app_sim ql_app_power)
    if(QL_APP_FEATURE_FILE_ZIP)
        target_link_libraries(${target} PRIVATE ql_app_zip)
    endif()
    if(QL_APP_FEATURE_FTP)
        target_link_libraries(${target} PRIVATE ql_app_ftp)
    endif()
    if(QL_APP_FEATURE_HTTP)
        target_link_libraries(${target} PRIVATE ql_app_http)
    endif()
    if(QL_APP_FEATURE_MMS)
        target_link_libraries(${target} PRIVATE ql_app_mms)
    endif()
    if(QL_APP_FEATURE_MQTT)
        target_link_libraries(${target} PRIVATE ql_app_mqtt)
    endif()
    if(QL_APP_FEATURE_SSL)
        target_link_libraries(${target} PRIVATE ql_app_ssl)
    endif()
    if(QL_APP_FEATURE_PING)
        target_link_libraries(${target} PRIVATE ql_app_ping)
    endif()
    if(QL_APP_FEATURE_NTP)
        target_link_libraries(${target} PRIVATE ql_app_ntp)
    endif()
    if(QL_APP_FEATURE_LBS)
        target_link_libraries(${target} PRIVATE ql_app_lbs)
    endif()

    if(QL_APP_FEATURE_CTSREG)
        target_link_libraries(${target} PRIVATE ql_app_ctsreg)
    endif()

    if(QL_APP_FEATURE_SOCKET)
        target_link_libraries(${target} PRIVATE ql_app_socket)
    endif()

    if(QL_APP_FEATURE_AUDIO)
        target_link_libraries(${target} PRIVATE ql_app_audio)
        if(QL_APP_FEATURE_TTS)
            add_library(ql_tts_api STATIC IMPORTED)
            set_target_properties(ql_tts_api PROPERTIES IMPORTED_LOCATION ${SOURCE_TOP_DIR}/components/newlib/armca5/libql_api_tts.a)
            target_link_libraries(${target} PRIVATE ql_app_tts ql_tts_api ${libm_file_name})
        endif()
    endif()
```

Figure 7: Link New Application Feature Component and Add Option Switch Control

After the application feature component is added, configure the relevant configuration files, and then compile the firmware according to ***Chapter 5***.

4.2.6. Feature Component Compilation Switch

QuecOpen CSDK provides *ql_app_feature_config.cmake* and *ql_app_feature_config.h.in* under the *components/ql-application* directory to facilitate configuring or removing features on the App side. *ql_app_feature_config.cmake* configures the feature components compiled by the compilation script;

ql_app_feature_config.h.in generates macros for enabling components. The generated macros will be used in the code. Configuration or removal of a certain feature requires simultaneous modifications in both files (see ***Chapter 6*** for details on feature removal).

The content of *ql_app_feature_config.cmake* is shown in the following figure. If a feature component is set to "**ON**", it indicates that the feature component will be compiled when the compilation script is executed; If it is set to "**OFF**", it indicates that the feature component will not be compiled when the compilation script is executed.

```
message("\n")

option(QL_APP_FEATURE_FTP  "Enable FTP" ON)
message(STATUS "QL_APP_FEATURE_FTP ${QL_APP_FEATURE_FTP}")

option(QL_APP_FEATURE_HTTP  "Enable HTTP" ON)
message(STATUS "QL_APP_FEATURE_HTTP ${QL_APP_FEATURE_HTTP}")

option(QL_APP_FEATURE_MQTT  "Enable MQTT" ON)
message(STATUS "QL_APP_FEATURE_MQTT ${QL_APP_FEATURE_MQTT}")

option(QL_APP_FEATURE_SSL  "Enable SSL" ON)
message(STATUS "QL_APP_FEATURE_SSL ${QL_APP_FEATURE_SSL}")

option(QL_APP_FEATURE_FILE  "Enable FILE" ON)
message(STATUS "QL_APP_FEATURE_FILE ${QL_APP_FEATURE_FILE}")

option(QL_APP_FEATURE_AUDIO  "Enable AUDIO" ON)
message(STATUS "QL_APP_FEATURE_AUDIO ${QL_APP_FEATURE_AUDIO}")
if(QL_APP_FEATURE_AUDIO)
option(QL_APP_FEATURE_AUDIO_TTS  "Enable TTS" ON)
else()
option(QL_APP_FEATURE_AUDIO_TTS  "Enable TTS" OFF)
endif()
message(STATUS "QL_APP_FEATURE_AUDIO_TTS ${QL_APP_FEATURE_AUDIO_TTS}")


option(QL_APP_FEATURE_WIFISCAN  "Enable WIFI-Scan" ON)
message(STATUS "QL_APP_FEATURE_WIFISCAN ${QL_APP_FEATURE_WIFISCAN}")

option(QL_APP_FEATURE_BT  "Enable BT" ON)
message(STATUS "QL_APP_FEATURE_BT ${QL_APP_FEATURE_BT}")

option(QL_APP_FEATURE_GPS  "Enable GPS" ON)
message(STATUS "QL_APP_FEATURE_GPS ${QL_APP_FEATURE_GPS}")


option(QL_APP_FEATURE_LCD  "Enable LCD" ON)
message(STATUS "QL_APP_FEATURE_LCD ${QL_APP_FEATURE_LCD}")

option(QL_APP_FEATURE_CAMERA  "Enable CAMERA" ON)
message(STATUS "QL_APP_FEATURE_CAMERA ${QL_APP_FEATURE_CAMERA}")
```

Figure 8: Configure Compilation Switch

The content of *ql_app_feature_config.h.in* is shown in the following figure. After adding the compilation switch of feature components in *ql_app_feature_config.cmake*, refer to the following figure to add a macro of the corresponding feature component:

```
#ifndef _QL_APP_FEATURE_CONFIG_H_
#define _QL_APP_FEATURE_CONFIG_H_

// @AUTO_GENERATION_NOTICE@

/**
 * whether to eanble APP feature FTP
 */
#cmakedefine QL_APP_FEATURE_FTP

/**
 * whether to eanble APP feature HTTP
 */
#cmakedefine QL_APP_FEATURE_HTTP

/**
 * whether to eanble APP feature MQTT
 */
#cmakedefine QL_APP_FEATURE_MQTT

/**
 * whether to eanble APP feature SSL
 */
#cmakedefine QL_APP_FEATURE_SSL

/**
 * whether to eanble APP feature FILE
 */
#cmakedefine QL_APP_FEATURE_FILE

/**
 * whether to eanble APP feature AUDIO
 */
#cmakedefine QL_APP_FEATURE_AUDIO

/**
 * whether to eanble APP feature TTS
 * if TTS is eanbled, you need enable AUDIO too
 */
#cmakedefine QL_APP_FEATURE_AUDIO_TTS
```

Figure 9: Add Macro Control

5 Application Compilation

5.1. Compilation Description

The *components\ql-config\download\pack* directory in QuecOpen CSDK contains a folder named after the module model that holds the current default Kernel firmware, the original application firmware stored by Quectel, and the relevant preset files such as *.map* and *.elf* files, as well as the firmware containing Kernel and application.

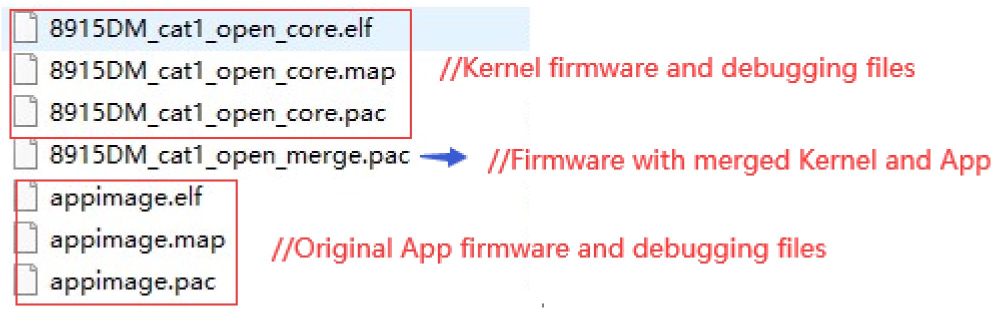


Figure 10: CSDK Original Firmware

The application source code is in the *components\ql-application* directory and is compiled by using the *build_all.bat* compilation script file (on Windows host system) or *build_all.sh* (on Linux host system) in the root directory of QuecOpen CSDK. Both script files use the same command parameters, but *build_all.sh* should be executed by **bash** command, i.e., **bash build_all.sh**. The application compilation process is explained below using the Windows host system as an example.

5.2. Compilation Procedure

1. Open CLI or PowerShell (only Windows 10 supports PowerShell) and execute **build_all.bat -h** to query the usage instructions. Common commands:

   - Compile command: **build_all <r/new> <Project> <Version>[VOLTE] [DSIM] [debug/release]**

     <r/new> Compilation type. r indicates incremental compilation and new indicates new compilation.

<Project>	Quectel module model in use. This parameter must match the folder name in the *components\ql-config\download\pack* directory.
<Version>	Name of compiled application firmware, which can be customized.
[VOLTE]	VoLTE feature (optional). VOLTE or NOVOLTE. Default: NOVOLTE.
[DSIM]	Single/dual card (optional). SINGLESIM: single card. DOUBLESIM: dual card. Default: SINGLESIM.
[debug/release]	Version (optional). A debug firmware is generated if you choose *debug* and an official releasable firmware is generated if you choose *release*. Default: release.

- Clears files generated during the previous compilation: **build_all clean**.

2. In CLI or PowerShell (only Windows 10 supports PowerShell), enter QuecOpen CSDK root directory and execute **build_all <r/new> <Project> <Version>[VOLTE] [DSIM] [debug/release]** to start compiling.

   After the command is executed, the compilation script first checks whether the *components\ql-config\download\pack* directory in QuecOpen CSDK contains a directory named **<Project>.** If yes, the compilation script continues compiling; If no, an error is reported and the compilation fails. The execution results are shown in the following figure:

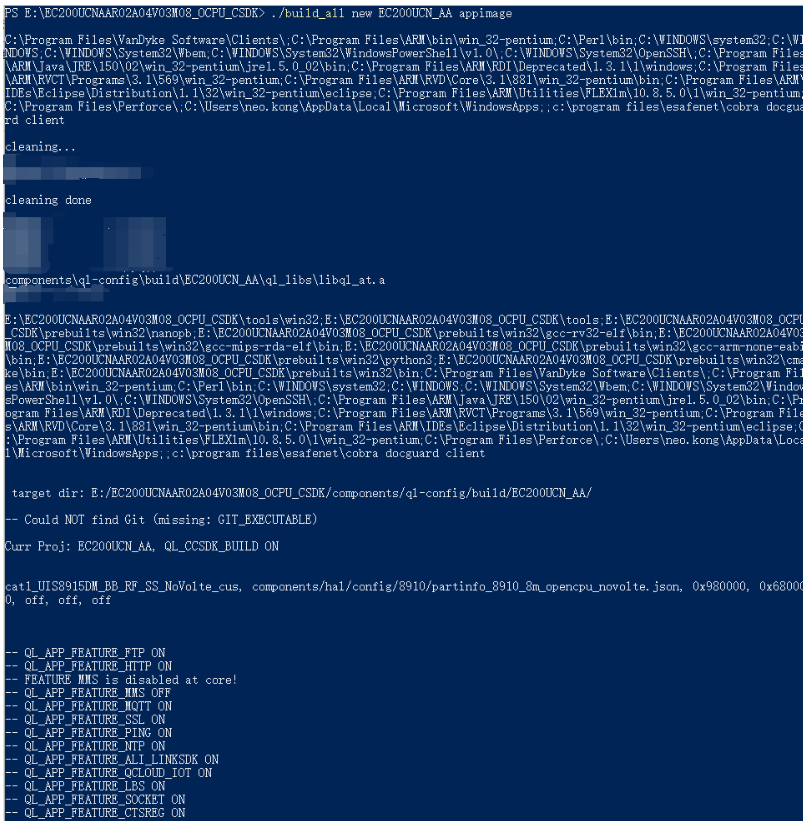


Figure 11: Compile Application

5.3. Compilation Result

During the compilation process, an *out* directory is generated in the QuecOpen CSDK root directory to store the generated compilation files.

After a successful compilation, a *target* directory is generated in the QuecOpen CSDK root directory, which stores the compiled Kernel firmware, target application firmware, and firmware containing the Kernel and application. If only the application firmware needs updating, burn the newly generated application firmware; If both the application and Kernel firmware need updating, burn the newly-generated firmware package containing both the Kernel and application.

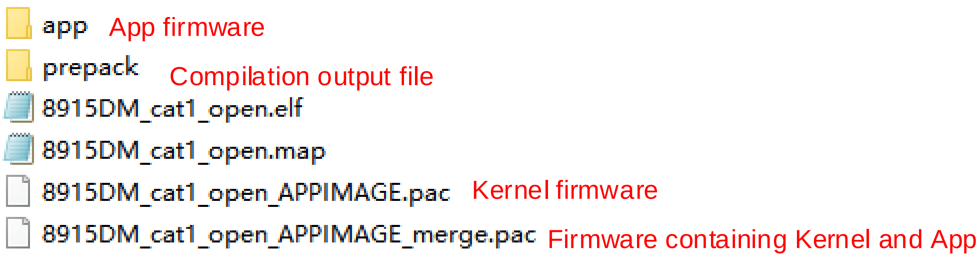


Figure 12: Compilation Results

5.4. Compiled File Clearance

Open a CLI or PowerShell (only Windows 10 supports PowerShell) and execute **build_all clean** to clear the files compiled in the *out* directory in QuecOpen CSDK. This action will not delete the target firmware in the *target* directory. If the compilation command parameters match, the firmware generated in the previous compilation version stored in the *target* directory is automatically cleared before the current compilation operation is successfully completed. If the compilation command parameters are different, the firmware generated in the previous compilation version stored in the *target* directory will not be cleared.

6 Feature Removal

6.1. Feature Removal in Kernel

QuecOpen CSDK opens all feature components by default, and the removable libraries are listed in *CMakeLists.txt* in the root directory. Each library is removed according to the macro control of kernel feature components, but removing the basic libraries is not allowed. You can remove the required feature in *components\ql-config\build\{project model}\8915DM_cat1_open\target.config*. A new compilation is required after removal, and Kernel firmware uses the files generated after compilation. See the following table for the removable libraries.

Figure 13: Removable Libraries in Kernel

You can see the following modification configuration file *target.config* for feature removal:

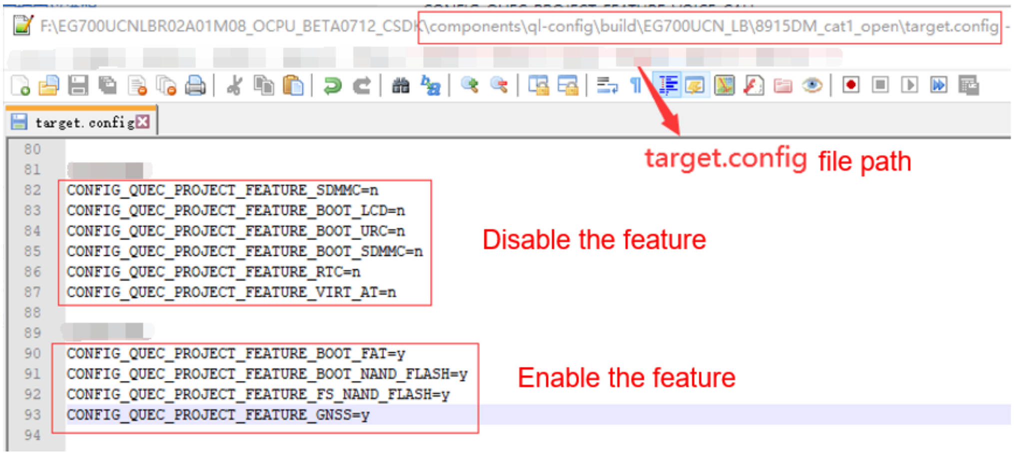


Figure 14: Feature Removal in Kernel

The removable libraries in Kernel are presented in the table below.

Table 3: Removable Libraries in Kernel

Feature Macro	Feature	Dependency
CONFIG_QUEC_PROJECT_FEATURE_USBNET	USBnet	-
CONFIG_QUEC_PROJECT_FEATURE_GNSS	GNSS	CONFIG_QUEC_PROJECT_FEATURE_UART
CONFIG_QUEC_PROJECT_FEATURE_RTC	RTC	-
CONFIG_QUEC_PROJECT_FEATURE_VIRT_AT	Virtual AT port	-
CONFIG_QUEC_PROJECT_FEATURE_FOTA	FOTA	-
CONFIG_QUEC_PROJECT_FEATURE_SMS	SMS	CONFIG_QUEC_PROJECT_FEATURE_HTTP
CONFIG_QUEC_PROJECT_FEATURE_VOICE_CALL	Voice call	-
CONFIG_QUEC_PROJECT_FEATURE_FTP	FTP	-
CONFIG_QUEC_PROJECT_FEATURE_HTTP	HTTP	-
CONFIG_QUEC_PROJECT_FEATURE_SSH2	SSH2	-
CONFIG_QUEC_PROJECT_FEATURE	MQTT	-

_MQTT		
CONFIG_QUEC_PROJECT_FEATURE_PING	Ping	-
CONFIG_QUEC_PROJECT_FEATURE_NTP	NTP	-
CONFIG_QUEC_PROJECT_FEATURE_MMS	MMS	-
CONFIG_QUEC_PROJECT_FEATURE_LBS	LBS	CONFIG_QUEC_PROJECT_FEATURE_HTTP
CONFIG_QUEC_PROJECT_FEATURE_UART	UART	-
CONFIG_QUEC_PROJECT_FEATURE_I2C	I2C	-
CONFIG_QUEC_PROJECT_FEATURE_KEYPAD	Matrix keypad	-
CONFIG_QUEC_PROJECT_FEATURE_LCD	LCD	-
CONFIG_QUEC_PROJECT_FEATURE_BT	Bluetooth	-
CONFIG_QUEC_PROJECT_FEATURE_BT_HFP	Bluetooth HFP	CONFIG_QUEC_PROJECT_FEATURE_BT
CONFIG_QUEC_PROJECT_FEATURE_BT_A2DP_AVRCP	A2DP	CONFIG_QUEC_PROJECT_FEATURE_BT
CONFIG_QUEC_PROJECT_FEATURE_BLE_GATT	BLE GATT	CONFIG_QUEC_PROJECT_FEATURE_BT
CONFIG_QUEC_PROJECT_FEATURE_CAMERA	Camera	-
CONFIG_QUEC_PROJECT_FEATURE_SPI	SPI	-
CONFIG_QUEC_PROJECT_FEATURE_SPI_FLASH	Single SPI flash (4-wire bus)	CONFIG_QUEC_PROJECT_FEATURE_SPI
CONFIG_QUEC_PROJECT_FEATURE_SPI_NOR_FLASH	Single SPI NOR flash (4-wire bus)	CONFIG_QUEC_PROJECT_FEATURE_SPI CONFIG_QUEC_PROJECT_FEATURE_SPI_FLASH
CONFIG_QUEC_PROJECT_FEATURE_SPI_NAND_FLASH	Single SPI NAND flash (4-wire bus)	CONFIG_QUEC_PROJECT_FEATURE_SPI CONFIG_QUEC_PROJECT_FEATURE_SPI_FLASH
CONFIG_QUEC_PROJECT_FEATURE_SDMMC	SDMMC	-
CONFIG_QUEC_PROJECT_FEATURE_PBK	PBK	-

CONFIG_QUEC_PROJECT_FEATURE_FILE_ZIP	File compression	-
CONFIG_QUEC_PROJECT_FEATURE_FS_NAND_FLASH	Single SPI NAND flash (4-wire bus) (file system)	CONFIG_QUEC_PROJECT_FEATURE_SPI CONFIG_QUEC_PROJECT_FEATURE_SPI_FLASH CONFIG_QUEC_PROJECT_FEATURE_SPI_NAND_FLASH
CONFIG_QUEC_PROJECT_FEATURE_SPI6_EXT_NOR	6-wire SPI NOR flash (file system)	-
CONFIG_QUEC_PROJECT_FEATURE_FILE_AT	File system AT command	-
CONFIG_QUEC_PROJECT_FEATURE_CLOUDOTA	HTTP OTA	CONFIG_QUEC_PROJECT_FEATURE_HTTP
CONFIG_QUEC_PROJECT_FEATURE_VOLTE	VoLTE	-
CONFIG_QUEC_PROJECT_FEATURE_AUDIO	Audio	-
CONFIG_QUEC_PROJECT_FEATURE_USB	USB	-
CONFIG_QUEC_PROJECT_FEATURE_LEDCFG	LED & PWM	-
CONFIG_QUEC_PROJECT_FEATURE_WIFISCAN	Wi-Fi Scan	-

6.2. Feature Removal in BootLoader

The following features in BootLoader are removable. If a feature is not required, disable it in *target.config*; If a feature is required, enable the *target.config* feature and then call the feature initialization function in *boot2_start_8910.c*. New compilation is required after feature removal.

Table 4: Removable Libraries in BootLoader

Feature Macro	Feature	Dependency
CONFIG_QUEC_PROJECT_FEATURE_BOOT_LCD	Boot LCD	CONFIG_QUEC_PROJECT_FEATURE_LCD
CONFIG_QUEC_PROJECT_FEATURE_BOOT_FAT	Boot FAT file system	-
CONFIG_QUEC_PROJECT_FEATURE_BOOT_SDMMC	Boot SDMMC	CONFIG_QUEC_PROJECT_FEATURE_BOOT_FAT

CONFIG_QUEC_PROJECT_FEATURE_BOOT_NAND_FLASH	Boot NAND Flash	CONFIG_QUEC_PROJECT_FEATURE_BOOT_FAT

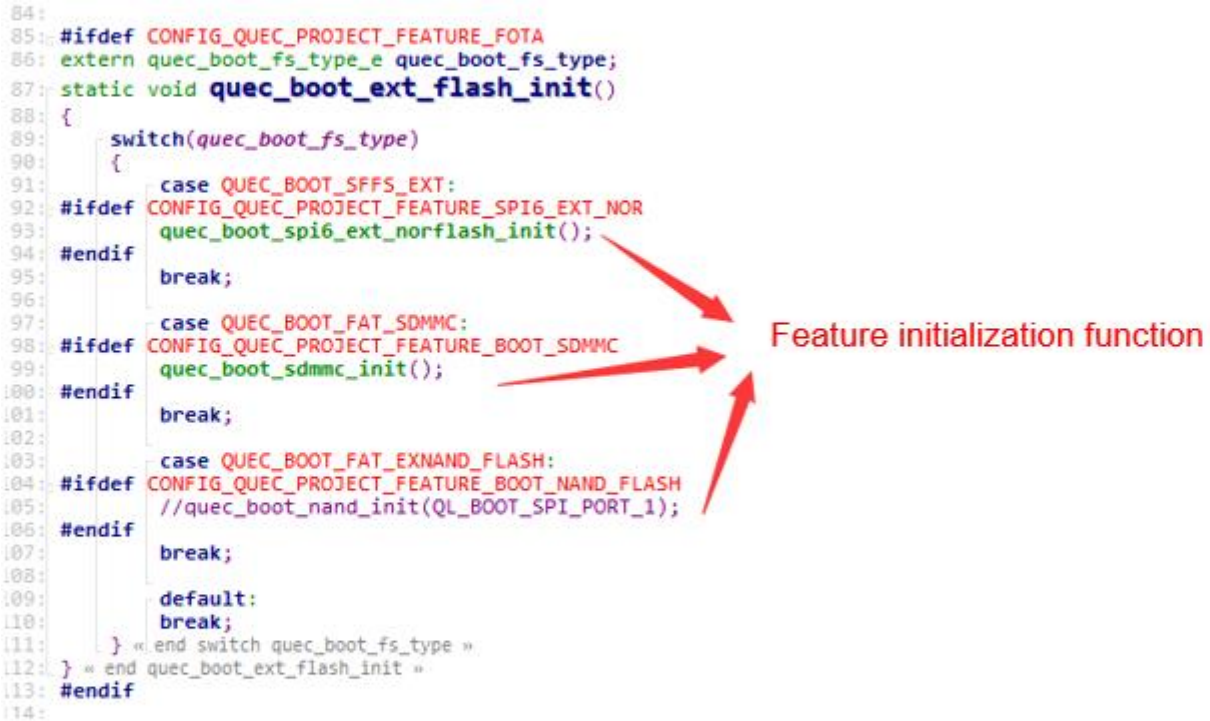


Figure 15: Feature Initialization Function

7 Preset File

A preset file is a designated initial file within a firmware package, which is written directly to the module file system during firmware downloading. Preset files take up file system space, and the maximum size of a single preset file is currently limited to 512 KB. If file size exceeds 512 KB, the file is skipped during downloading.

7.1. Enable Preset File Packing

In the *ql_app_feature_config.cmake* file, you can configure whether to pack the preset file through QL_APP_PACK_FILE.:

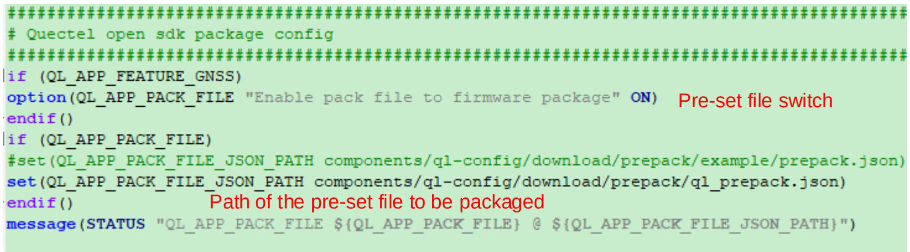


Figure 16: Preset File Packing Compilation Configuration

NOTE

The preset file packing compilation configuration is affected by the GNSS in SDK. When GNSS is enabled, the GNSS chip firmware (currently about 238 KB) will be packed into the APP firmware as a preset file by default. For details about the GNSS supported by the EC600U series, ECx00G family and EG800G series modules, see the hardware design documents corresponding to each module.

If other feature files need to be preset, you should add the corresponding feature switch. For example, if you want to preset the audio file, add a switch for audio.

```
##################################################################
# Quectel open sdk package config
##################################################################
if (QL_APP_FEATURE_GNSS OR QL_APP_FEATURE_AUDIO)
option(QL_APP_PACK_FILE "Enable pack file to firmware package" ON)
endif()
```

Figure 17: Add Feature Switch for Preset File

7.2. Preset File Configuration

QL_APP_PACK_FILE_JSON_PATH in *ql_app_feature_config.cmake* file specifies the configuration path of a specific preset file. The script packs the preset file according to the specified *json* file. The *json* file in CSDK specifies the path of the GNSS chip firmware, which is represented by a relative path, i.e., it is in the same path where the *json* file is stored. You can refer to this format in the *json* file to add and modify the preset file in the "files" node in the figure below. Be careful not to preset too many files to avoid taking up too much file system space.

```
{
    "_comment": [
        "This is the configuration file to pre-pack files to pac.",
        "It contains a list of _files_, and for each file,",
        "_file_ is the absolute path in target, and _local_file_",
        "is the path in host. When _local_file_ is relative path,",
        "it is related to the directory of this configuration file.",
        "Also @SOURCE_TOP_DIR@ and @BINARY_TOP_DIR@ will be substitued",
        "to the absolte of current build.",
        "",
        "When it is empty, PREPACK won't be inserted into pac."
    ],
    "files": [
        {
            "file": "/user/boot",
            "local_file": "bootloader_r3.0.0_build2817_uartboot_4468750.pkg"
        },
        {
            "file": "/user/firm",
            "local_file": "UC6226CI-R3.2.0.12Build6815_mfg.pkg"
        }
    ]
}
```

Path and name of the specified file

The path where the file is located

Figure 18: Preset File Packing Configuration File

The preset files are usually downloaded to the *user* directory, which is convenient for querying via the module FILE API. Note that the total length of the file path and file name to be written cannot exceed 192 bytes.

7.3. Preset File Downloading and Upgrading

The preset file can also be downloaded, upgraded or deleted through FOTA.

- Upgrade the preset file in the old firmware version to that in the new firmware version.
- If the old firmware version does not have a preset file, you can add a preset file by FOTA.
- If the old firmware version has a preset file and the new firmware version does not have a preset file, the preset files will be deleted after FOTA upgrade;
- If the preset files need to be ignored in FOTA upgrade, you can modify the value of *method* to "ignore" from "diff" in <paccpio id="PREPACK" method="diff"> </paccpio> of xml configuration file. See ***document [2]*** for details.

```
</pacnv>
<paccpio id="PREPACK" method="diff">
    <!--
        <file name="some_file_name" method="ignore"/>
    -->
</paccpio>
```

Figure 19: Ignore Preset File in FOTA

8 Flash Partition Adjustment

Quectel EC600U series, ECx00G family and EG800G series modules support adjusting embedded flash partitions, including Kernel, application and file system partition size. See ***documents [2]*** and ***[3]*** for specific implementation.

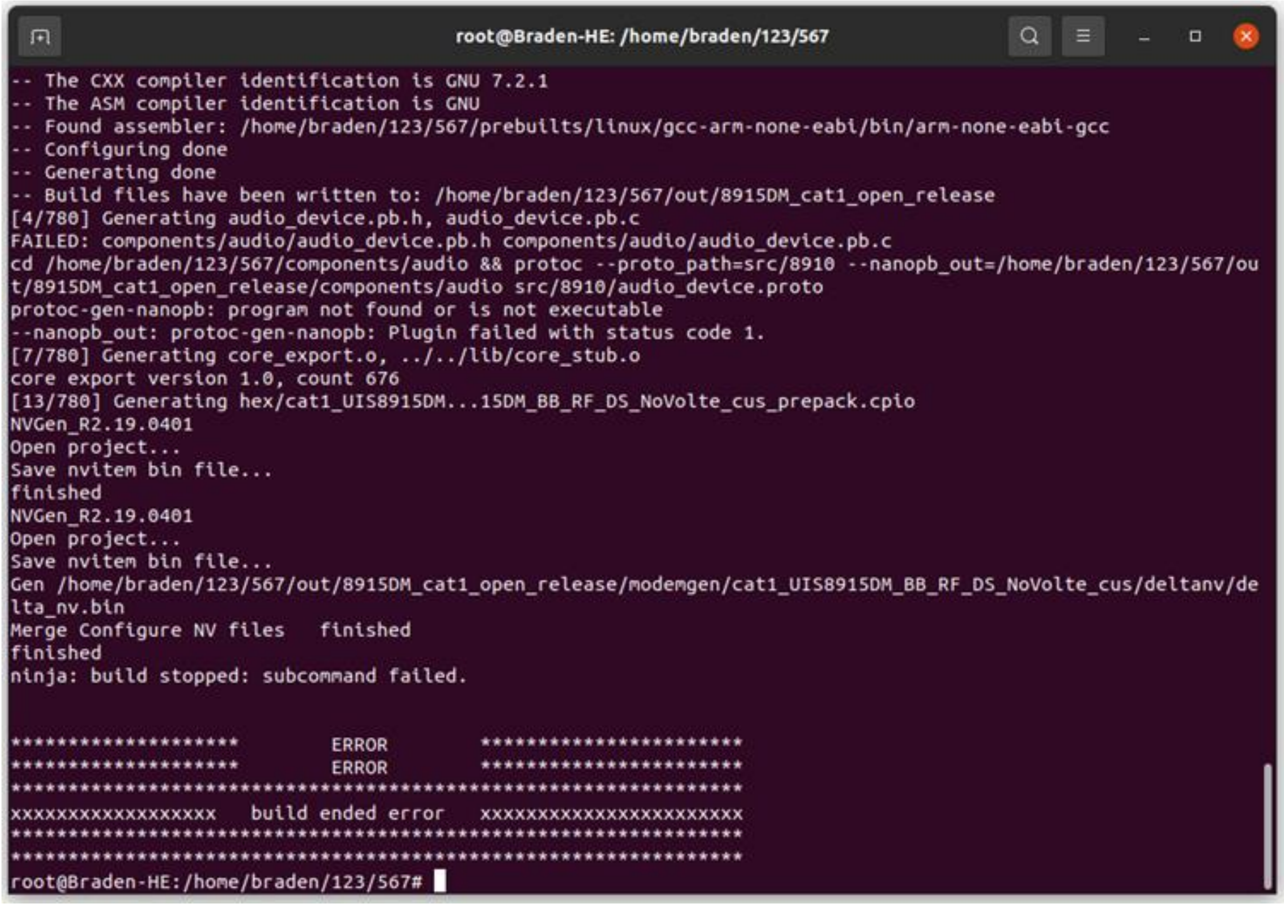
9 Firmware Burning

Quectel EC600U series, ECx00G family and EG800G series modules support burning firmware with QFlash. See ***document [4]*** for QFlash use.

10 FAQ

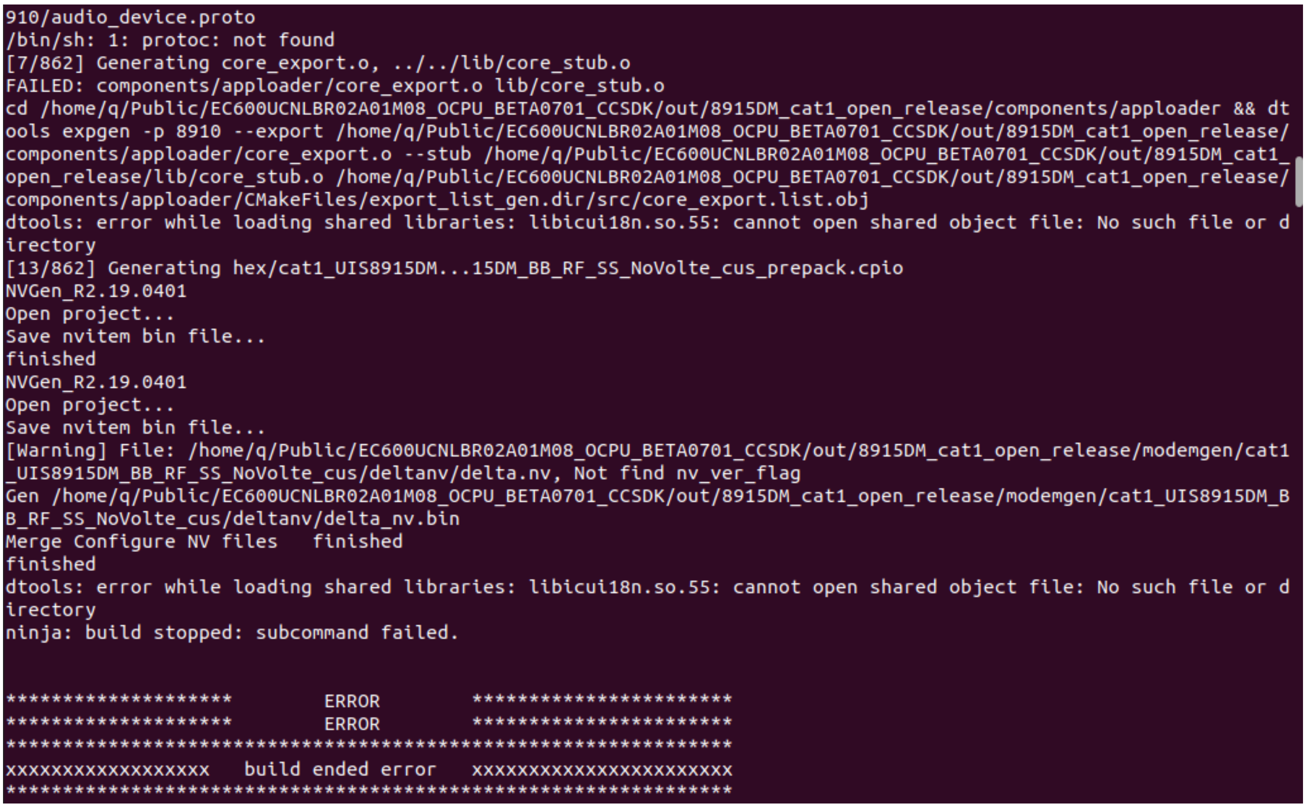
10.1. Common Compilation Error in Linux

1. The error shown in the figure below indicates that the required software is not installed on the host system. Just reinstall the software and see ***Chapter 3.1*** for details.



2. The error message "dtools: error while loading shared libraries: libicui18n.so.55:canot openShared object file:No such file or directory. Ninja:build stopped:subcommand failed" indicates that dtools is missing a component. You can install the component by executing the following command in sequence.

```
wget http://security.ubuntu.com/ubuntu/pool/main/i/icu/libicu55_55.1-7_amd64.deb
sudo dpkg -i libicu55_55.1-7_amd64.deb
```



```
910/audio_device.proto
/bin/sh: 1: protoc: not found
[7/862] Generating core_export.o, ../../lib/core_stub.o
FAILED: components/apploader/core_export.o lib/core_stub.o
cd /home/q/Public/EC600UCNLBR02A01M08_OCPU_BETA0701_CCSDK/out/8915DM_cat1_open_release/components/apploader && dt
ools expgen -p 8910 --export /home/q/Public/EC600UCNLBR02A01M08_OCPU_BETA0701_CCSDK/out/8915DM_cat1_open_release/
components/apploader/core_export.o --stub /home/q/Public/EC600UCNLBR02A01M08_OCPU_BETA0701_CCSDK/out/8915DM_cat1_
open_release/lib/core_stub.o /home/q/Public/EC600UCNLBR02A01M08_OCPU_BETA0701_CCSDK/out/8915DM_cat1_open_release/
components/apploader/CMakeFiles/export_list_gen.dir/src/core_export.list.obj
dtools: error while loading shared libraries: libicui18n.so.55: cannot open shared object file: No such file or d
irectory
[13/862] Generating hex/cat1_UIS8915DM...15DM_BB_RF_SS_NoVolte_cus_prepack.cpio
NVGen_R2.19.0401
Open project...
Save nvitem bin file...
finished
NVGen_R2.19.0401
Open project...
Save nvitem bin file...
[Warning] File: /home/q/Public/EC600UCNLBR02A01M08_OCPU_BETA0701_CCSDK/out/8915DM_cat1_open_release/modemgen/cat1
_UIS8915DM_BB_RF_SS_NoVolte_cus/deltanv/delta.nv, Not find nv_ver_flag
Gen /home/q/Public/EC600UCNLBR02A01M08_OCPU_BETA0701_CCSDK/out/8915DM_cat1_open_release/modemgen/cat1_UIS8915DM_B
B_RF_SS_NoVolte_cus/deltanv/delta_nv.bin
Merge Configure NV files   finished
finished
dtools: error while loading shared libraries: libicui18n.so.55: cannot open shared object file: No such file or d
irectory
ninja: build stopped: subcommand failed.


********************        ERROR        ***********************
********************        ERROR        ***********************
****************************************************************
xxxxxxxxxxxxxxxxxx   build ended error   xxxxxxxxxxxxxxxxxxxxxxx
****************************************************************
```

11 Appendix References

Table 5: Related Documents

Document Name
[1] Quectel_EC200D&ECx00G&EC600U&EG800G_Series_QuecOpen_FOTA_API_Reference_Manual
[2] Quectel_EC600U_Series_QuecOpen(SDK)_Embeded_Flash_Partition_Adjustment_Guide
[3] Quectel_ECx00G&EG800G_Series_QuecOpen(SDK)_Embeded_Flash_Partition_Adjustment_Guide
[4] Quectel_QFlash_User_Guide

Table 6: Terms and Abbreviations

Abbreviation	Description
A2DP	Advanced Audio Distribution Profile
API	Application Programming Interface
App	Application
BLE	Bluetooth Low Energy
BT	Bluetooth
CLI	Command-line Interface
FAT	File Allocation Table
FOTA	Firmware Over-The-Air
FTP	File Transfer Protocol
GCC	GNU Compiler Collection
GNSS	Global Navigation Satellite System
GPIO	General-Purpose Input/Output

HFP	Hands-free Profile
HTTP	Hypertext Transfer Protocol
I2C	Inter-Integrated Circuit
IoT	Internet of Things
LBS	Location Based Services
LCD	Liquid Crystal Display
LED	Light Emitting Diode
MMS	Multimedia Messaging Service
MQTT	Message Queuing Telemetry Transport
NTP	Network Time Protocol
OSI	Open System Interconnection Reference Model
PBK	Phonebook
PWM	Pulse Width Modulation
RTC	Real Time Clock
RTOS	Real-Time Operating System
SDK	Software Development Kit
SDMMC	Secure Digital/Multimedia Card
SMS	Short Message Service
SPI	Serial Peripheral Interface
SSH	Secure Shell
UART	Universal Asynchronous Receiver/Transmitter
USB	Universal Serial Bus
(U)SIM	(Universal) Subscriber Identity Module
VoLTE	Voice (voice calls) over LTE
Wi-Fi	Wireless Fidelity